

Milestone 5 –Tracabilite et securisation des operations

Sprint V5 - Al Qalam Stock Manager Theme : decorateurs, functools.wraps, protocole descripteur, dataclass

Objectif

Ajouter une couche transversale de validation et de journalisation automatique sans modifier le corps des methodes metier. Les decorateurs s'intercalent comme des filtres. Les descripteurs sur Produit rendent impossible la creation d'un produit avec des donnees invalides.

Ce qui change par rapport a V4

```
alqalamv5/
├── decorateurs/
│   ├── journalisation.py  <- @journaliser(operation) --
decorateur avec parametre
│   ├── validation.py      <- @chronometre @valider_qte @valider_ref
├── models/
│   ├── descripteurs.py    <- NOUVEAU : Positif, PositifEntier, NonVide
│   ├── produit.py        <- utilise les descripteurs (validation automatique)
├── services/
│   ├── journal_service.py <- NOUVEAU : EntreeJournal (@dataclass) + JournalService
│   └── stock_service.py  <- methodes decorees avec @journaliser + @valider_qte
```

Concepts mis en oeuvre

Concept	Fichier	Ce que ca illustre
Decorateur simple	validation.py	Wrapping sans parametre
Decorateur avec parametre	journalisation.py	Fabrique de decorateurs (3 niveaux)
@functools.wraps	journalisation.py	Preserve name, doc
Double decoration	stock_service.py	Ordre d'execution, capture des erreurs
Protocole descripteur	descripteurs.py	get, set, set_name
@dataclass	journal_service.py	Generation automatique de init

Etape 1 –Decorateur simple : @chronometre

```
import functools, time

def chronometre(func):
    @functools.wraps(func)    # TOUJOURS : preserve __name__, __doc__, __qualname__
    def wrapper(*args, **kwargs):
        debut = time.perf_counter()
        resultat = func(*args, **kwargs)
        print(f"[chrono] {func.__qualname__} -> {(time.perf_counter()-debut)*1000:.2f}")
        return resultat
    return wrapper
```

Pourquoi @functools.wraps est obligatoire :

```
# Sans @wraps
@chronometre
def sauvegarder(self): "Sauvegarde le stock."

print(sauvegarder.__name__) # "wrapper"          INCORRECT
print(sauvegarder.__doc__) # None                INCORRECT

# Avec @functools.wraps(func)
print(sauvegarder.__name__) # "sauvegarder"      OK
print(sauvegarder.__doc__) # "Sauvegarde..."  OK
```

Etape 2 –Decorateur avec parametre : @journaliser("entree")

```
@journaliser("entree")
def entree_stock(self, ref, qte, note=""):
    ...

# Equivalent exact :
# entree_stock = journaliser("entree")(entree_stock)

def journaliser(operation: str):    # NIVEAU 1 -- fabrique, recoit "entree"

    def decorateur(func):          # NIVEAU 2 -- recoit la fonction decoree

        @functools.wraps(func)
        def wrapper(self, *args, **kwargs): # NIVEAU 3 -- execute a chaque appel
            debut = datetime.now()
            try:
```

```

        resultat = func(self, *args, **kwargs)
        return resultat
    except Exception:
        raise
    finally:
        # finally : s'exécute TOUJOURS, même si exception levée
        duree_ms = int((datetime.now() - debut).total_seconds() * 1000)
        if hasattr(self, "_journal"):
            self._journal.enregistrer(operation=operation, ...)

    return wrapper

return decorateur # La fabrique retourne le vrai decorateur

```

Etape 3 –Double decoration et ordre d'exécution

```

@journaliser("entree") # decorateur EXTERNE (enveloppe externe)
@valider_qte(min_val=1) # decorateur INTERNE (enveloppe interne)
def entree_stock(self, ref, qte, note=""):
    ...

```

Ordre à l'appel avec qte=50 valide : 1. wrapper_journaliser démarre 2. wrapper_valider_qte valide qte=50 3. entree_stock s'exécute 2. wrapper_valider_qte retourne 1. wrapper_journaliser enregistre SUCCES

Ordre à l'appel avec qte=-5 invalide : 1. wrapper_journaliser démarre 2. wrapper_valider_qte lève ValueError 1. wrapper_journaliser finally -> enregistre ECHEC -> ValueError propagée vers l'UI

Le journal capture toutes les tentatives, valides ou non.

Etape 4 –Protocole descripteur sur Produit

```

class Positif:
    def __set_name__(self, owner, name):
        # Appele une fois a la construction de la classe
        self._nom_privé = f"_{name}" # "prix_achat" -> "_prix_achat"

    def __get__(self, obj, objtype=None):
        if obj is None: return self # acces via Produit.prix_achat
        return getattr(obj, self._nom_privé, 0)

    def __set__(self, obj, valeur):
        # Appele a CHAQUE ecriture : produit.prix_achat = 1.5

```

```

    valeur = float(valeur)
    if valeur < 0:
        raise ValueError(f"Valeur negative interdite : {valeur}")
    setattr(obj, self._nom_privé, valeur)

```

```

class Produit:
    # Descripteurs au niveau de la CLASSE -- interceptent toute affectation
    prix_achat = Positif()
    prix_vente = Positif()
    qte        = PositifEntier()
    seuil_min  = PositifEntier()
    ref        = NonVide()
    nom        = NonVide()
    categorie  = NonVide()

```

Avantage vs @property : un descripteur est réutilisable sur N attributs de N classes. Avec @property il faut copier le code pour chaque attribut.

Etape 5 –@dataclass pour EntreeJournal

```

from dataclasses import dataclass, field
from datetime import datetime

@dataclass
class EntreeJournal:
    # @dataclass genere __init__, __repr__, __eq__ automatiquement
    operation : str
    ref       : str
    succes    : bool
    erreur    : str = ""
    duree_ms  : int  = 0
    # field(default_factory=...) : valeur calculée a chaque instantiation
    timestamp : datetime = field(default_factory=datetime.now)

e = EntreeJournal("entree", "CRAY-001", True, duree_ms=3)
print(e)
# EntreeJournal(operation='entree', ref='CRAY-001', succes=True, duree_ms=3, ...)

```

Criteres de validation

- ☐ py -3 main.py lance l'app avec 4 onglets sans erreur
- ☐ Produit avec prix negatif -> ValueError levee par le descripteur

- ☐ Entree stock -> apparait dans l'onglet Journal (icone , succes=True)
 - ☐ Sortie avec qte superieure au stock -> erreur visible dans le journal (icone )
 - ☐ stock.entree_stock.__name__ retourne "entree_stock"(pas "wrapper")
 - ☐ Filtre "Erreurs seulement" ne montre que les echecs
 - ☐ Bouton "Effacer journal" vide le tableau
-

Pour aller plus loin

- Decorateur @tentative(n=3) qui relance la methode N fois avant de lever l'exception
 - Decorateur @mettre_en_cache (equivalent de @functools.lru_cache)
 - Sauvegarder le journal dans data/journal.log avec rotation automatique
 - Utiliser inspect.signature pour capturer les noms des parametres dans le log
-

Al Qalam Stock Manager –Formation Python Partie II –Sprint V5